# Автоматическая генерация признаков против ручного FE в ранжировании KION

**T-Lab 2026 · RecSys · Задание 1**

## Коротко

Построили ранжировщик LightGBM с ручным FE на данных KION и сравнили его с четырьмя версиями автоматической генерации признаков. Все пайплайны обучены, отобраны и проверены на одном и том же сплите: 20 000 пользователей для обучения, 40 000 для валидации и 40 000 для теста, одинаковые даты и кандидаты. На тесте автоматический V2 лучше ручного FE (NDCG@10 0.142 против 0.136; против ручного набора, улучшенного по абляциям, отрыв меньше, +0.004), а V4, который добавляет к V2 признаки совместных просмотров (co-visitation), даёт ещё +0.005 и становится лучшим набором. Поиск формул V3 ручной FE не обогнал.

В работах других авторов на полном датасете KION NDCG@10 достигает 0.166–0.198 (eSASRec 0.166 и HSTU 0.169 на RecSys 2025, гибрид с демографическими признаками 0.198 в ВКР ВШЭ 2026). Мы обучаемся на меньшем объёме данных и по другому протоколу (другое окно цели, пул кандидатов вместо всего каталога), поэтому напрямую эти метрики не сравнимы, но они показывают теоретический максимум, к которому можно стремиться на этих данных. Подробные результаты - в разделе 3.5, выводы - в разделе 4.

## 1. Изучили датасет

**Что в данных.** MTS KION, полный архив: 5.48 млн взаимодействий, 962 тыс. пользователей, 15.7 тыс. объектов, период 13.03–22.08.2021. К взаимодействиям прилагаются метаданные пользователей (возраст, доход, пол, наличие детей) и объектов (тип, жанры, страны, режиссёры, студии, год, рейтинг, `for_kids`).

**Что выяснилось при изучении и как это повлияло на решение.**

1. **Одна строка - это агрегированная пара user–item, а не событие.** В ней хранится дата *последнего* просмотра, суммарная длительность и `watched_pct`. Повторных взаимодействий нет, поэтому прямые признаки «сколько раз пользователь смотрел этот объект» на кандидатах всегда равны нулю, и вся работа ложится на косвенные связи через атрибуты. Пары, которые начали смотреть до cutoff и досмотрели после, целиком выпадают из истории и выглядят как новые объекты.
2. **Показов и явных отказов нет.** Отрицательный пример - это кандидат, которого пользователь не посмотрел, а не наблюдённый отказ.
3. **Активность за неделю разреженная.** Из 40 000 случайных пользователей с историей в следующую неделю что-то смотрят 4 200–4 500, то есть 10–11%. Поэтому выборки нужны большие, иначе интервалы получаются слишком широкими.
4. **Популярность доминирует.** Популярность за 7 дней на тесте даёт NDCG@10 0.127, а коллаборативный ItemKNN только 0.088. Свежая популярность объекта будет главным признаком любой модели, а обогнать её в топ-10 трудно.
5. **Метаданные статичны.** Исторических версий нет, поэтому они считаются известными на любой момент времени.
6. **Данные заканчиваются 22.08.2021.** Последняя полная неделя 16–23.08 - единственный кандидат на тест, поэтому независимость теста обеспечивается не новым временем, а новыми пользователями.

**Постановка, которая из этого следует.**

- **Задача:** для пары (user, item) на момент `cutoff` предсказать, появится ли запись user–item в следующие 7 дней.
- **Метка 1:** запись с `cutoff ≤ last_watch_dt < cutoff + 7 дней`. **Метка 0:** кандидат без такой записи.
- **История** для признаков: только строки с `last_watch_dt < cutoff`. Это проверяется тестами, которые удаляют и подменяют будущие строки и требуют неизменности признаков.

**Единый сплит для всех пайплайнов.** Пользователи с историей случайно перемешаны и разбиты на блоки по 20 000; пользователи ранних пилотных прогонов исключены. Все наборы признаков, все поиски и все абляции используют одни и те же блоки и даты:

| | **Пользователи** | **Cutoff** | **Целевая неделя** | **Оцениваемых пользователей** | **Пар** |
|---|---|---|---|---|---|
| Обучение | 20 000 | 12.07, 19.07, 26.07, 02.08 | до 09.08 | 2 075–2 436 в неделю | 906 тыс. |
| Валидация | 40 000 других | 09.08 | 09.08–16.08 | 4 514 | 451 тыс. |
| **Тест** | **40 000 других** | 16.08 | 16.08–23.08 | **4 212** | 421 тыс. |

- Блоки не пересекаются; тестовые пользователи раньше нигде не использовались.
- Валидация - единственное место для early stopping, калибровки Платта и любого выбора: поиска окон, формул и семейств признаков, абляций.
- Генераторы признаков обучаются (словари, пороги, схемы) только на истории обучающих пользователей.
- До открытия теста сохранён план: финальные наборы признаков и список объявленных контрастов. Тест открыт один раз, после сохранения всех моделей.

**Кандидаты** (общие для всех моделей): до 100 объектов на пользователя - 50 из ItemKNN (cosine по бинарной матрице user–item, 50 соседей) и 50 самых популярных за 7 дней. Уже просмотренное исключено, положительные пары принудительно не добавляются. Кандидаты строятся внутри каждого блока из 20 000 пользователей. **Скоры кандидатов** - 4 признака: ранг популярности, скор и ранг ItemKNN, флаг источника.

**Потолок кандидатов.** В пул попадает 31.5% целевых пар (в среднем по пользователям 39%), у 44% тестовых пользователей нет ни одного положительного кандидата, и даже идеальная сортировка пула даёт NDCG@10 не выше 0.429. На валидации картина та же: 31.2%, 44.5%, 0.421.

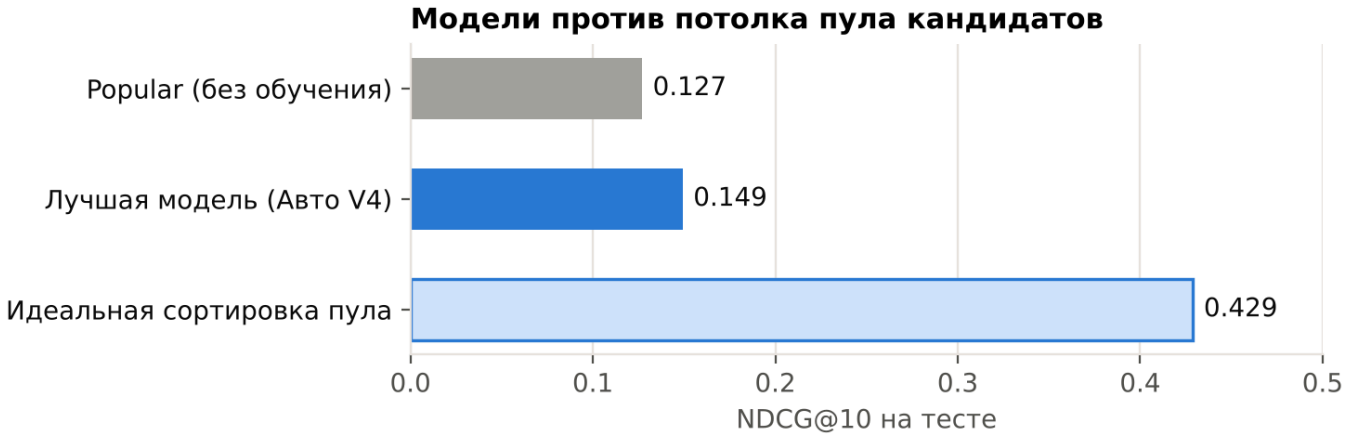



*Рисунок 1. NDCG@10 на тесте: популярность, лучшая модель и идеальная сортировка того же пула кандидатов.*

**Метрики.** NDCG@10 и Recall@10 считаются по **полному** целевому набору пользователя (пропущенные кандидатами объекты тоже в знаменателе) и усредняются по пользователям;

пользователи без положительных кандидатов остаются с нулём. ROC-AUC и LogLoss считаются по всем парам user–candidate. Для LambdaRank LogLoss считается после калибровки Платта на валидации; дополнительно для каждого набора обучена бинарная модель (`objective=binary`). Интервалы - парный bootstrap по тестовым пользователям, 2 000 повторов.

**Модель.** LightGBM (`learning_rate=0.05`, `num_leaves=15`, `min_child_samples=50`, `reg_lambda=1`), early stopping по валидации (до 1 000 деревьев, терпение 50). Финальные модели обучены с 5 bagging-seed (subsample и colsample 0.8), предсказания усреднены. Поисковые модели - один seed без bagging.

## 2. Сделали решение бустингом с ручным FE

**Ручные признаки** (`manual`, 66 признаков; ядро `manual_core` - 18):

- **Пользователь:** демография; активность за 3/7/14/30/90 дней и всю историю; давность, тренд, средний досмотр и его разброс, доля досмотров >80%, средняя длительность, энтропия жанров, доля сериалов.
- **Объект:** тип, год, рейтинг, число жанров; популярность за 1/3/7/14/30 дней и всю историю, доли в общей активности, тренды 3/7 дней, давность, возраст в каталоге, сглаженный досмотр.
- **Связи user–item через атрибуты:** доли жанров в окнах 7/14/30/все и их максимум, затухание с полураспадом 14 дней, давность и новизна жанра, жанровый lift с поправкой на частоту, совпадение жанров с последними днями; предпочтения типа контента, стран и режиссёров; разница года и рейтинга со средними пользователя; популярность объекта в возрастной группе и по полу.

**Результат на тесте.** Ручной FE без скоров кандидатов даёт NDCG@10 0.128 - на уровне популярности (0.127, разница +0.001, CI [−0.004; +0.006]), хотя по ROC-AUC разрыв огромный: 0.818 против 0.757. Модель лучше упорядочивает весь пул, но её топ-10 почти совпадает с популярным. Скоры кандидатов поднимают ручной FE до 0.136 (+0.008, CI [+0.005; +0.012]). Это и есть сильный бейзлайн: **ручной + скоры, NDCG@10 0.1364, ROC-AUC 0.824**. Около 58% его gain приходится на популярность объекта (`i_count7`, `i_count3`, доля в популярности за 7 дней), 20% - на скоры кандидатов, остальное - на жанровый lift, страны и сегментную популярность.

**Абляции** (на валидации, удаляется по одной группе из модели «ручной + скоры»):

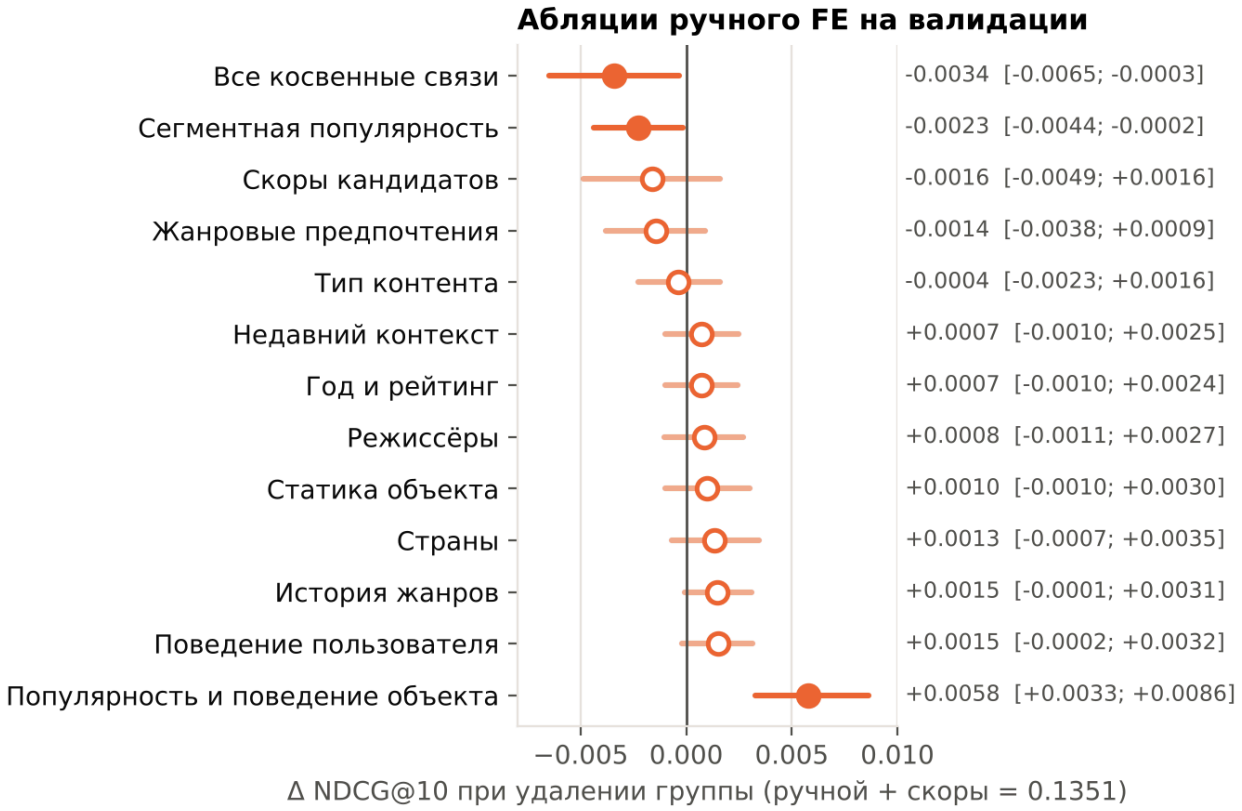



*Рисунок 2. Изменение NDCG@10 на валидации при удалении группы признаков. Закрашенные точки - интервал не включает ноль.*

- Значимо нужны **косвенные связи через атрибуты** целиком (−0.0034) и **сегментная популярность** (−0.0023). По отдельности жанры, страны, режиссёры и прочие подгруппы взаимозаменяемы, и ни одна не значима.
- Демография пользователя деревьями не используется вовсе: модель без неё идентична полной.
- Неожиданно: без группы **популярности и поведения объекта** NDCG растёт на 0.0058 (значимо). Эти 17 признаков дублируют скоры кандидатов и перетягивают на себя модель. Чтобы сравнение с V2 было симметричным (V2 тоже выбирался по валидации), мы обучили и улучшенный ручной вариант без этой группы - «ручной без популярности объекта», 49 признаков (+4 скора). Результаты - в разделе 3.5.

## 3. Проверили четыре версии автогенерации

Автогенерация - главная часть работы. Задача генератора - построить признаки для ранжировщика без ручного списка: на вход он получает только таблицы и роли трёх полей (пользователь, объект, время), а всё остальное выводит сам.

**Общая схема всех версий.**

1. **Профилирование схемы.** На истории обучающих пользователей до первого cutoff генератор определяет типы полей: числовые поля событий (`total_dur`, `watched_pct`), числовые метаданные (год, рейтинг, `for_kids`), категориальные поля с небольшим числом значений (возраст, доход, пол, наличие детей) и списковые атрибуты объекта, через которые можно связать пользователя и объект (жанры, страны, режиссёры, студии, тип контента). Атрибуты выбираются по покрытию и размеру словаря, без взгляда на метки. Словари, пороги и выбранные поля замораживаются в схеме.
2. **Генерация кандидатов-признаков** по грамматике конкретной версии.

3. **Чистка на обучении:** удаляются константы и точные дубликаты.
4. **Отбор** (в V2–V4) - только по валидации; тест в выборе не участвует.
5. **Вычисление** для любого cutoff строго по истории `last_watch_dt < cutoff`. Тесты проверяют, что удаление и подмена будущих строк не меняют ни одного значения.

Каждая версия отвечала на недостаток предыдущей: V1 - фиксированная грамматика, V2 - поиск окон и новых операций, V3 - поиск произвольных формул, V4 - сигнал из графа user–item, которого нет в агрегатах по атрибутам.

**Таблица 1. Генераторы в одном месте**

| | V1 | V2 | V3 | V4 |
|---|---|---|---|---|
| Что перебирает | ничего, грамматика фиксирована | окна × семейства операций | деревья выражений | семейства графовых признаков поверх V2 |
| Размер пространства | 60 рецептов | 20 конфигураций, до 97 рецептов в каждой | 215 формул | 4 семейства, до 33 признаков |
| Как выбирает | - | NDCG@10 на валидации | gain на обучении + размер набора по валидации | жадно по валидации, bootstrap P > 0.8 |
| Итог | 54 признака | 92 признака | 120 формул | 102 признака (V2 + 10) |
| Время поиска | < 1 с | 563 с | 116 с | 165 с |
| Тест, NDCG@10 со скорами | 0.1384 | 0.1415 | 0.1376 | 0.1466 |

### 3.1. V1: признаки из схемы

**Грамматика.** Признак - это сочетание «сущность × агрегат × поле × окно × свёртка». Генератор перебирает все допустимые сочетания:

| Сущность | Агрегаты | Пример признака |
|---|---|---|
| объект, пользователь | число событий в окне 7 / 30 дней / вся история; среднее `total_dur` и `watched_pct` в окне; давность последнего события | сколько раз объект смотрели за 7 дней |
| пара user–item | число событий в окне | (на KION всегда 0, удаляется чисткой) |
| атрибут объекта в истории пользователя | доля событий пользователя с этим атрибутом в окне 7 / 30 дней, свёртка mean или max по атрибутам кандидата; давность последнего события с атрибутом | доля жанров кандидата среди просмотров пользователя за 30 дней |
| числовой вкус | модуль разницы числового поля объекта и среднего этого поля в истории пользователя | насколько год выпуска кандидата отличается от привычного пользователю |
| метаданные | сырые числовые и закодированные категориальные поля | возраст пользователя, тип контента |

Из 60 рецептов после чистки осталось 54 признака. Окна 7 и 30 дней заданы заранее, никакого отбора нет.

**Результат на тесте:** 0.1280 без скоров и 0.1384 со скорами, то есть на уровне ручного FE (+0.002, CI [−0.001; +0.006]). По ROC-AUC V1 значимо хуже ручного (−0.004). Самые важные

признаки - число просмотров объекта за 7 и 30 дней (42% и 15% gain), доля жанров и стран кандидата в истории пользователя, разница года выпуска и `for_kids` со вкусом пользователя.

**Чего не хватало.** Окна заданы руками, генератор не отвечает на вопрос «почему 7 дней, а не 10». Доли атрибутов не учитывают, насколько атрибут популярен вообще: доля драм 40% у любителя драм и у среднего пользователя значит разное. Нет сегментной популярности, которая в абляциях ручного FE оказалась значимой.

### 3.2. V2: поиск окон и семейств операций

**Новые семейства.** К базе V1 добавлены три семейства операций, каждое строится по той же схеме таблиц ($\alpha = 10$ - сила сглаживания к общему среднему):

| Семейство | Операции | Что означает |
|---|---|---|
| **dynamics** | тренд: $\frac{n_{[t-w,\,t)}+1}{n_{[t-2w,\,t-w)}+1}$ для пользователя и объекта; лог-среднее и разброс длительности; сглаженное среднее досмотра объекта $\frac{\sum x+\alpha\bar{x}}{n+\alpha}$ | растёт ли интерес к объекту, насколько стабильно его досматривают |
| **affinity** | сглаженный lift атрибута $\frac{c_{u,a}+\alpha p_a}{(n_u+\alpha)p_a}$; доля атрибута с экспоненциальным затуханием (полураспад = короткое окно); средний досмотр пользователя по атрибуту | во сколько раз пользователь смотрит жанр чаще, чем в среднем по всем; свежие предпочтения весят больше |
| **segments** | сглаженный lift объекта в сегменте пользователя $\frac{c_{s,i}+\alpha p_i}{(n_s+\alpha)p_i}$ для каждого категориального поля пользователя (возраст, доход, пол) | насколько объект популярнее среди «таких же» пользователей, чем в целом |

Здесь $c_{u,a}$ - число просмотров пользователя $u$ с атрибутом $a$, $n_u$ - все просмотры пользователя, $p_a$ - доля атрибута во всех просмотрах; для сегмента $s$ и объекта $i$ - аналогично. Сглаживание не даёт редким атрибутам и маленьким сегментам получить огромный lift по одному-двум просмотрам.

**Поиск.** Короткое окно перебиралось среди 3, 7, 10 и 14 дней (длинное - 30), семейства - пять вариантов: только база V1, база + одно семейство (три варианта), все семейства. Для каждой из 20 конфигураций генератор обучался заново, признаки чистились, ранжировщик обучался на 4 неделях и оценивался на валидации.

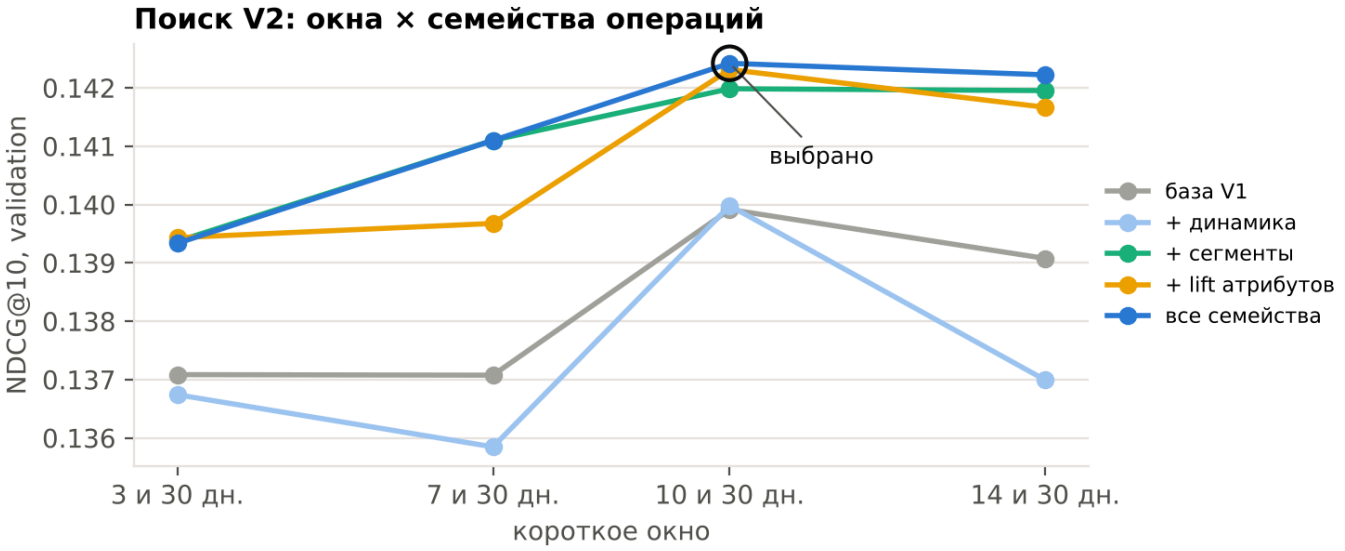


*Рисунок 3. NDCG@10 на валидации для каждого сочетания окон и семейств.*

**Что показал поиск.**

- **Окно 10 дней лучше 7 при любом наборе семейств** (база V1: 0.1399 против 0.1371). Ручной FE и V1 использовали 7 дней по привычке; поиск нашёл более удачное окно для недельного горизонта.
- **Сглаженный lift атрибутов и сегментная популярность дают основной прирост:** +0.002–0.004 к базе при каждом окне. Это ровно те два сигнала, которых не хватало V1.
- **Динамика сама по себе не помогает** (на окнах 3, 7 и 14 даже хуже базы), но не мешает в составе всех семейств.
- Выбрана конфигурация «окна 10 и 30 дней, все семейства»: 92 признака, валидация 0.1424.

**Результат на тесте:** 0.1381 без скоров и 0.1415 со скорами. **V2 + скоры лучше ручного + скоры: Δ NDCG@10 +0.0051 [+0.0021; +0.0082]**, ROC-AUC +0.004 и LogLoss −0.0003 тоже значимо лучше. Без скоров разрыв вдвое больше: +0.0101 [+0.0066; +0.0136].

**Какие признаки работают** (доля gain в модели без скоров): число просмотров объекта за 10 дней (37%) и 30 дней (12%), сглаженный lift жанров (9%), стран и типа контента (по 3%), тренд объекта за 10 дней, разница `for_kids` со вкусом пользователя, сегментный lift по полу. Новые семейства V2 вместе дают около трети gain.

**Почему V2 обогнал ручной FE.** Признаки V2 по смыслу похожи на ручные (популярность в окне, жанровые предпочтения, популярность по возрасту и полу), но генератор строит их **систематически**: сглаженный lift - для всех пяти атрибутов, а не только для жанров; сегментный lift - для всех трёх категориальных полей пользователя (возраст, доход, пол); окно популярности подобрано по данным. Кроме того, генератор сам нашёл `studios` и `for_kids`, которых нет в ручном наборе.

### 3.3. V3: поиск формул

**Идея.** Вместо фиксированной грамматики - язык выражений, из которого можно собрать любую формулу заданной сложности (до 24 узлов, глубина до 10). Формула - это дерево:

| Тип узла | Операторы |
|---|---|
| источник | `events` (вся история до cutoff) |
| операции над таблицей | `window(days, offset)` - окно 3/7/10/14/30 дней или предыдущий такой же период; `filter(field, quantile)` - только события выше квантиля 0.5 / 0.8; `join_attribute(field)` - развернуть события по атрибутам объекта; `decay(half_life)` - экспоненциальные веса; `map_log` - логарифм поля |
| агрегаты | `count`, `sum`, `mean`, `std`, `last_age` по пользователю, объекту, паре «пользователь × атрибут» или глобально; свёртка mean / max / min по атрибутам кандидата |
| арифметика | `divide`, `add`, `multiply`, `abs_diff`, `log1p` |
| готовые блоки | `attribute_lift`, `segment_item_lift` (как в V2) |

Каждое выражение проверяется на типы (табличные операторы - только над таблицами, арифметика - над скалярами) и компилируется в безопасное вычисление без `eval`.

**Поиск в четыре шага.**

1. **Исходные формулы (104):** счётчики и средние по пользователю и объекту в окнах, разброс, давность, доли атрибутов в истории пользователя, разница числовых метаданных со вкусом пользователя, счётчики по событиям с высоким досмотром.
2. **Мутации (64):** 12 лучших формул по gain пробной модели (80 деревьев на обучении) мутируются - меняется окно, добавляется фильтр по квантилю, счётчик заменяется на среднее или разброс, строится отношение текущего окна к предыдущему (тренд), меняется свёртка.
3. **Обогащение (47):** комбинации операторов, которых не было среди исходных: лог-среднее, сглаженное среднее досмотра, доля атрибута с затуханием, attribute lift по всем атрибутам.
4. **Отбор:** после удаления 6 дубликатов в пуле 209 формул. Они ранжируются по gain пробной модели на обучении, а размер набора выбирается по валидации: 24, 48, 80, 120 формул или весь пул. Лучшим оказались 120 формул (0.1392 на валидации; весь пул - 0.1384).

Примеры формул из итогового набора:

```
count BY item OVER events[last 7d]
count BY item OVER events[last 3d]
attribute_lift(genres, alpha=10) OVER events
attribute_lift(countries, alpha=10) OVER events
std(total_dur) BY item OVER events[last 30d]
```

**Результат на тесте:** 0.1330 без скоров и 0.1376 со скорами. От ручного + скоры V3 по NDCG не отличается (+0.0013, CI [−0.0017; +0.0041]), а по ROC-AUC значимо хуже (−0.003). У бинарной модели V3 + скоры значимо лучше ручного (+0.006), так что вывод о V3 неустойчив к выбору функции потерь.

**Почему V3 не обогнал V2.** Сильнейшие формулы - `count BY item OVER events[last 7d]` (29% gain), то же за 3 дня (16%), `attribute_lift(genres)` (9%) и `attribute_lift(countries)` (3%). Это почти буквально ручные `i_count7`, `i_count3` и `ui_genre_lift`: поиск воспроизвёл решения инженера, но нового сигнала не нашёл. При этом отбор по gain на обучении не различает близкие формулы, и в 120 признаках много почти одинаковых вариантов одних и тех же счётчиков; ранжировщик делит между ними деревья, и качество получается ниже, чем у компактного V2 с подобранным окном и сглаживанием.

### 3.4. V4: совместные просмотры и другие графовые признаки

**Идея.** Все признаки V1–V3 связывают пользователя с кандидатом через атрибуты (жанр, страну, сегмент) или через популярность. Коллаборативный сигнал «какие объекты смотрят вместе» в пайплайне уже есть, но только в виде кандидатов ItemKNN и их скоров (один скор и

ранг на пару). V4 проверяет, даёт ли что-то более богатое представление этого графа в признаках. Все семейства строятся только по ролям user, item и время:

| Семейство | Признаков | Как считается |
|---|---|---|
| **co-visitation** | 10 | Косинусная близость объектов по бинарной матрице просмотров $S_{ij} = \frac{\lvert U_i \cap U_j \rvert}{\sqrt{\lvert U_i \rvert \lvert U_j \rvert}}$. Для кандидата $i$ - сумма $S_{ij}$ по истории пользователя с затуханием (полураспад 7 и 30 дней, в том числе нормированная), максимум, среднее, сумма, близость к последним 1 / 3 / 10 просмотрам |
| **recent co-visitation** | 3 | То же, но матрица близости строится только по последним 14 дням |
| **SVD** | 2 | Низкоранговая близость пользователя и объекта: разложение матрицы просмотров с idf-весами, ранг 64, скалярное произведение и косинус |
| **фильтрованная популярность** | 18 | Популярность объекта в окнах от 1 до 21 дня, только по досмотрам больше 50% и только среди пользователей, активных в последнюю неделю |

**Отбор.** Сначала каждое семейство добавлялось к V2 по отдельности, затем жадно в порядке одиночного прироста. Семейство оставлялось, если прирост на валидации положителен с bootstrap-вероятностью больше 0.8.

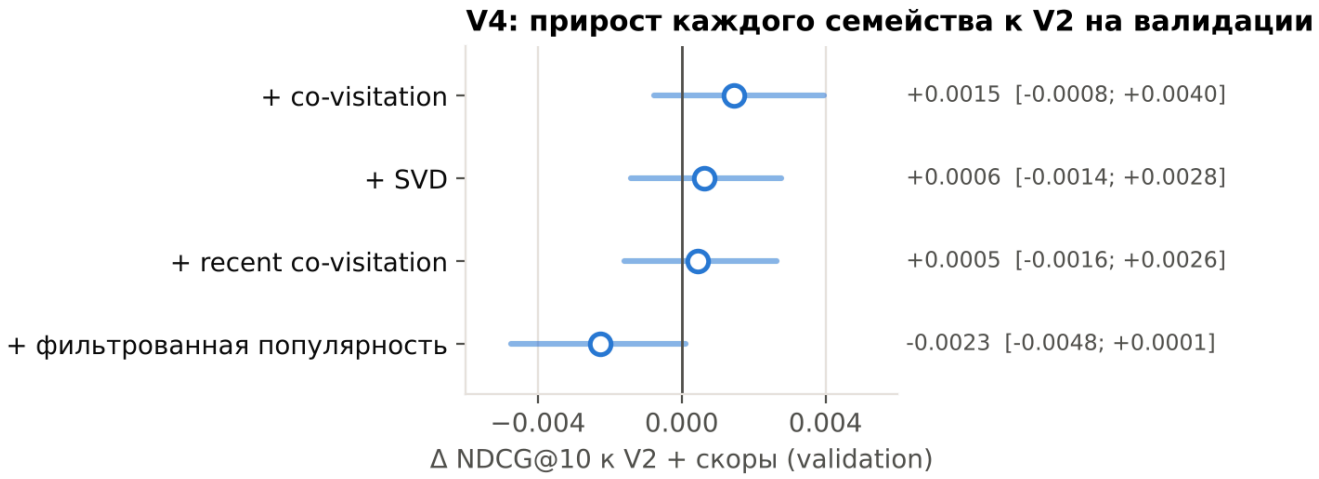



*Рисунок 4. Прирост каждого семейства к V2 на валидации.*

**Что выбрал отбор:** только co-visitation (+0.0015, P = 0.90). После него SVD добавил +0.0004 при P = 0.65 и был отброшен, recent co-visitation и фильтрованная популярность ухудшили модель. Итог - 102 признака.

**Результат на тесте: 0.1491 без скоров** (лучший NDCG@10 из всех наборов) и 0.1466 со скорами (лучшие ROC-AUC 0.834 и LogLoss). **V4 + скоры лучше V2 + скоры: +0.0051 [+0.0029; +0.0074]**, и значимо лучше по Recall@10, ROC-AUC и LogLoss. Co-visitation забирает 13% gain модели; самые полезные признаки - максимальная близость кандидата к истории пользователя, сумма с затуханием 30 дней и близость к последним 10 просмотрам.

**Почему сработало.** Co-visitation отвечает на вопрос «смотрят ли этот объект те же люди, что смотрели объекты из истории пользователя» - это коллаборативный сигнал. В грубом виде он уже есть в скоре ItemKNN, но co-visitation даёт более богатое представление: затухание по времени, максимум близости, близость к последним просмотрам. На тесте прирост оказался больше, чем на валидации у поисковой модели (+0.0051 против +0.0015). Вероятная причина - поисковые модели обучались с одним seed без bagging, а финальная - с усреднением 5 моделей, которое лучше использует 10 новых признаков. SVD несёт похожий, но более грубый сигнал и после co-visitation лишний; фильтрованная популярность дублирует популярность из V2 и скоров кандидатов.

### 3.5. Итоговое сравнение на тесте

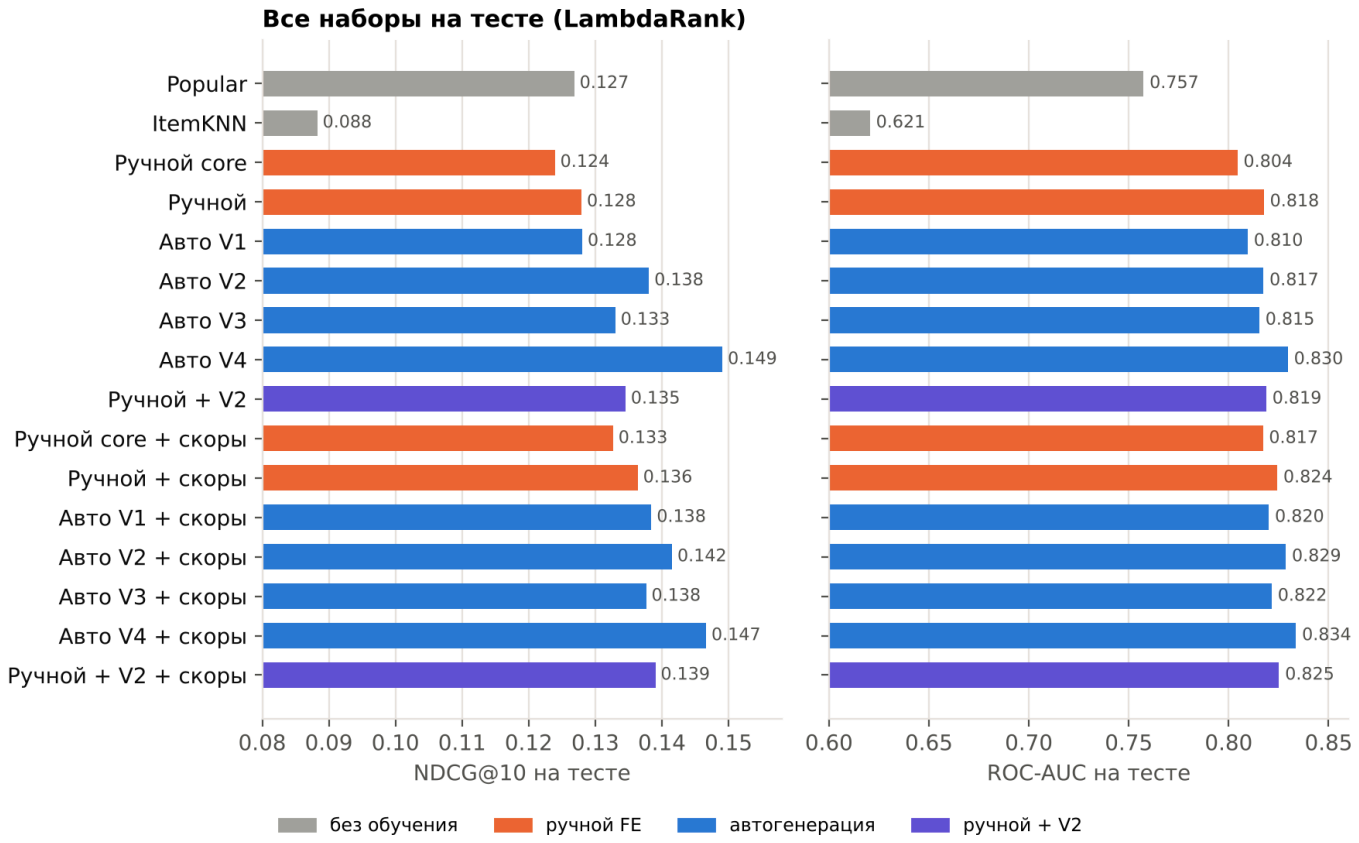



*Рисунок 5. NDCG@10 и ROC-AUC всех наборов на тесте.*

**Таблица 2. Тест: 4 212 оцениваемых пользователей из 40 000, неделя 16–23.08.2021 (LambdaRank, 5 seed)**

| Модель | Признаки | NDCG@10 | Recall@10 | ROC-AUC | LogLoss |
|---|---|---|---|---|---|
| Popular | 0 | 0.1268 | 0.2083 | 0.757 | 0.0454 |
| ItemKNN | 0 | 0.0883 | 0.1326 | 0.621 | 0.0471 |
| Ручной | 66 | 0.1279 | 0.2024 | 0.818 | 0.0413 |
| Ручной + скоры | 70 | 0.1364 | 0.2077 | 0.824 | 0.0409 |
| Ручной без популярности объекта | 49 | 0.1225 | 0.1813 | 0.795 | 0.0420 |
| Ручной без популярности объекта + скоры | 53 | 0.1379 | 0.2071 | 0.827 | 0.0407 |
| Авто V1 | 54 | 0.1280 | 0.2012 | 0.810 | 0.0416 |
| Авто V1 + скоры | 58 | 0.1384 | 0.2093 | 0.820 | 0.0410 |
| Авто V2 | 92 | 0.1381 | 0.2077 | 0.817 | 0.0412 |
| Авто V2 + скоры | 96 | 0.1415 | 0.2131 | 0.829 | 0.0406 |
| Авто V3 | 120 | 0.1330 | 0.1990 | 0.815 | 0.0414 |
| Авто V3 + скоры | 124 | 0.1376 | 0.2057 | 0.822 | 0.0410 |
| **Авто V4** | 102 | **0.1491** | **0.2221** | 0.830 | 0.0405 |
| Авто V4 + скоры | 106 | 0.1466 | 0.2198 | **0.834** | **0.0403** |
| Ручной + V2 | 158 | 0.1345 | 0.2053 | 0.819 | 0.0412 |
| Ручной + V2 + скоры | 162 | 0.1390 | 0.2080 | 0.825 | 0.0408 |

«+ скоры» - к набору добавлены 4 признака от генераторов кандидатов (ранг популярности, скор и ранг ItemKNN, флаг источника). Без скоров видно, чего стоят сами признаки набора, со скорами - итоговая модель. LogLoss у LambdaRank - после калибровки Платта на валидации. Бинарные модели в целом согласуются с LambdaRank, но порядок близких наборов местами меняется (например, бинарный V3 + скоры обгоняет V1 + скоры); лучший LogLoss - у бинарного V4 + скоры (0.0402).

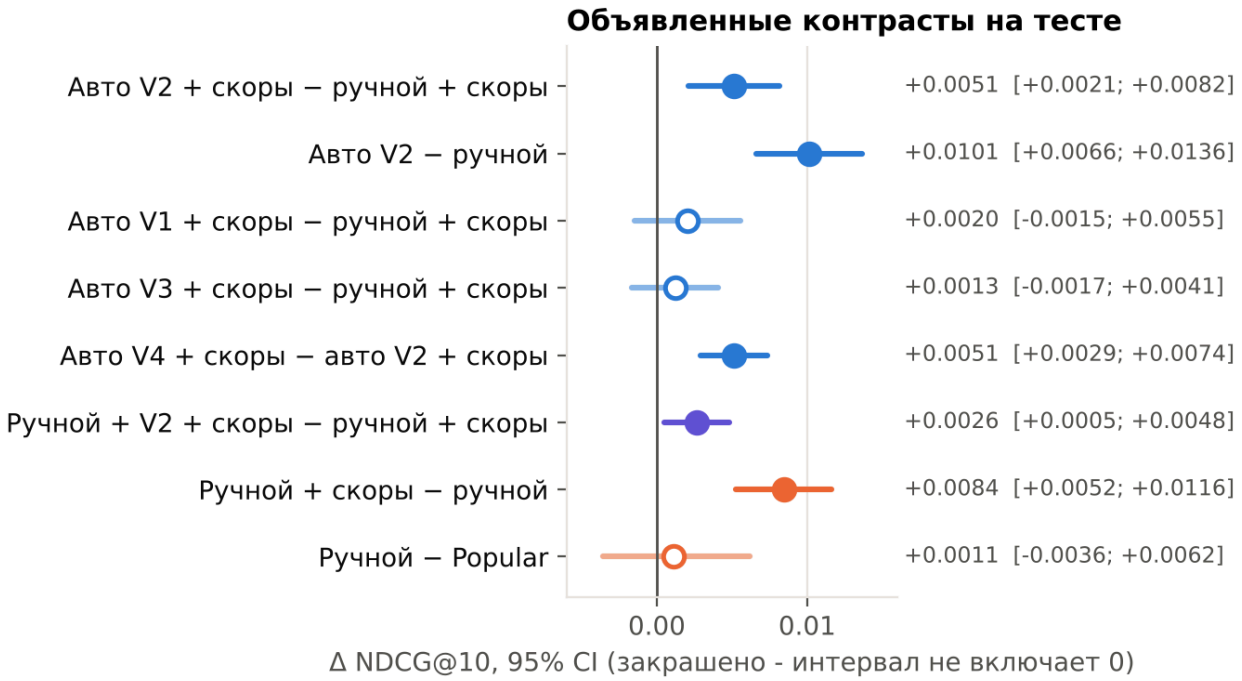


*Рисунок 6. Разности NDCG@10 на тесте с 95% CI для контрастов, объявленных до открытия теста.*

**Таблица 3. Объявленные контрасты на тесте (LambdaRank)**

| Контраст | Δ NDCG@10 [95% CI] | Δ Recall@10 [95% CI] | Δ ROC-AUC [95% CI] | Δ LogLoss [95% CI] |
|---|---|---|---|---|
| **V2 + скоры − ручной + скоры (главный)** | **+0.0051** [+0.0021; +0.0082] | +0.0053 [−0.0000; +0.0110] | **+0.0043** [+0.0023; +0.0063] | **−0.0003** [−0.0004; −0.0002] |
| V2 − ручной | **+0.0101** [+0.0066; +0.0136] | +0.0052 [−0.0007; +0.0112] | −0.0002 [−0.0033; +0.0028] | **−0.0002** [−0.0003; −0.0000] |
| V1 + скоры − ручной + скоры | +0.0020 [−0.0015; +0.0055] | +0.0016 [−0.0040; +0.0078] | **−0.0042** [−0.0074; −0.0011] | +0.0001 [−0.0000; +0.0003] |
| V3 + скоры − ручной + скоры | +0.0013 [−0.0017; +0.0041] | −0.0020 [−0.0066; +0.0025] | **−0.0028** [−0.0047; −0.0009] | **+0.0001** [+0.0000; +0.0002] |
| V4 + скоры − V2 + скоры | **+0.0051** [+0.0029; +0.0074] | **+0.0067** [+0.0029; +0.0107] | **+0.0051** [+0.0033; +0.0069] | **−0.0003** [−0.0004; −0.0002] |
| Ручной + V2 + скоры − ручной + скоры | **+0.0026** [+0.0005; +0.0048] | +0.0003 [−0.0032; +0.0038] | +0.0009 [−0.0006; +0.0023] | **−0.0001** [−0.0001; −0.0000] |
| Ручной + скоры − ручной | **+0.0084** [+0.0052; +0.0116] | **+0.0053** [+0.0002; +0.0102] | **+0.0067** [+0.0045; +0.0090] | **−0.0004** [−0.0005; −0.0003] |
| Ручной − Popular | +0.0011 [−0.0036; +0.0062] | −0.0058 [−0.0127; +0.0014] | **+0.0602** [+0.0505; +0.0706] | **−0.0041** [−0.0044; −0.0038] |

Главный контраст объявлен заранее как основной, остальные - вторичные. С бинарной функцией потерь основное преимущество V2 и V4 сохраняется (главный контраст +0.0077 [+0.0043; +0.0110]), и по NDCG@10 знаки всех контрастов те же. Но вторичные эффекты меняются: V3 + скоры против ручного + скоры у бинарной модели значимо лучше по NDCG (+0.0064 [+0.0033; +0.0096]), а по Recall, ROC-AUC и LogLoss меняет знак на противоположный (у V1 + скоры меняется знак LogLoss).

**Сравнение с улучшенным ручным набором.** Ручной без популярности объекта + скоры на тесте даёт 0.1379, это на +0.0015 выше полного ручного + скоры, но разница не значима ([−0.0012; +0.0042]). V2 + скоры лучше и его: +0.0036 [+0.0007; +0.0070] по NDCG@10 и +0.0060 [+0.0011; +0.0111] по Recall@10, хотя по ROC-AUC разница уже не значима. С бинарной функцией потерь отрыв V2 от этого варианта не значим (+0.0025 [−0.0007; +0.0060]). V4 + скоры лучше улучшенного ручного при обеих функциях потерь: +0.0088 [+0.0054; +0.0123] у LambdaRank и +0.0079 [+0.0046; +0.0116] у бинарной.

Отдельно проверили простое объединение ручного набора с V2 (158 признаков). Со скорами оно лучше ручного + скоры на +0.0026, но хуже одного V2 + скоры (0.1390 против 0.1415): наборы во многом дублируют друг друга, поэтому V2 скорее заменяет ручной FE, чем дополняет его.

### 3.6. Вычислительные затраты

**Таблица 4. Затраты на игровом ноутбуке (WSL2, 8 потоков)**

| Этап | Ручной | Авто V1 | Авто V2 | Авто V3 | Авто V4 |
|---|---|---|---|---|---|
| Построение признаков (обучение + валидация + тест) | 15.5 с | 9.2 с | 18.3 с | 26.0 с | 25.3 с |
| Поиск набора | дни ручной работы | - | 563 с (20 моделей) | 116 с | 165 с (поверх V2) |
| Обучение, 5 seed × (LambdaRank + binary), со скорами | 122 с | 138 с | 178 с | 179 с | 185 с |
| Инференс на тесте (421 тыс. пар) | 3.6 с | 4.2 с | 4.6 с | 4.3 с | 5.1 с |

- Кандидаты общие для всех наборов и стоят дороже любых признаков: 75 с на обучение, валидацию и тест вместе.
- Признаки любого набора строятся за секунды; разница между ручным и автоматическими наборами в обучении и инференсе - в пределах полутора раз.
- Весь эксперимент (кандидаты, 14 наборов, абляции, все поиски, тест, bootstrap) занял 62 минуты при пиковой памяти 14 ГБ.

## 4. Сделали выводы

1. **Автоматический FE лучше ручного.** Грамматика агрегатов V2 на независимом тесте обгоняет ручной набор при одинаковом входе: +0.005 NDCG@10 со скорами кандидатов и +0.010 без них, со значимо лучшими ROC-AUC и LogLoss. Против ручного набора, улучшенного по абляциям, отрыв меньше (+0.004 у LambdaRank, значимо; у бинарной модели не значимо), а V4 обгоняет и его (+0.009). Вероятная причина - V2 систематически строит сглаженные lift по всем атрибутам (жанры, страны, режиссёры, студии, тип) и сам подбирает окно популярности (10 дней вместо ручных 7).
2. **Лучший результат - у V4: агрегаты V2 плюс совместные просмотры.** Co-visitation добавляет к V2 ещё +0.005 NDCG@10 и улучшает все четыре метрики. Коллаборативный сигнал был и раньше в скорах ItemKNN, но более богатое его представление (затухание, максимум близости, последние просмотры) оказалось полезным - это единственное семейство, давшее прирост сверх V2.
3. **Сложнее не значит лучше.** Поиск формул V3 при большем числе признаков хуже V2. С LambdaRank он не показал преимущества над ручным FE, а с бинарной функцией потерь показал (+0.006), так что результат V3 зависит от функции потерь; его сильнейшие формулы почти буквально совпали с ручными. Фильтрованная популярность и SVD в V4 тоже ничего не добавили.
4. **Главное практическое преимущество автоматики - стоимость.** Поиск V2 и V4 занимает около 12 минут машинного времени против дней ручной итерации, а построение признаков стоит секунды.
5. **Ручной и автоматический наборы описывают во многом одни и те же сигналы.** Их объединение лучше ручного, но хуже V2 в одиночку: и инженер, и генератор приходят к популярности в окнах и жанровым связям.
6. **Резерв качества - в кандидатах, а не в ранжировщике.** Ручной FE без скоров не обгоняет популярность по NDCG@10, хотя по ROC-AUC разрыв огромный (0.818 против

0.757): модели лучше упорядочивают весь пул, но топ-10 в недельном горизонте почти совпадает с популярным. При потолке пула 0.429 и 44% пользователей без единого положительного кандидата следующий шаг - улучшать генерацию кандидатов, например теми же признаками co-visitation.

**Ограничения.**

1. **Агрегированный KION:** одна строка на пару с датой последнего просмотра, начало и повторы просмотров не восстановить. Это одинаково для всех моделей.
2. **Один период:** тест - последняя неделя KION; независимость обеспечена новыми пользователями, а не новым временем, поэтому эффекты конкретной недели (релизы, сезонность) не устранены.
3. **Метаданные** - статические снимки без истории версий.
4. **Одни гиперпараметры** для всех наборов; разброс по seed учтён bagging-усреднением 5 моделей, а поиски и абляции сделаны с одним seed.
5. **Метрики валидации оптимистичны:** на ней делались early stopping, калибровка и весь выбор; выводы опираются только на тест.
6. **Кандидаты и граф считаются внутри блока из 20 000 пользователей**, а не по всему KION; на полном датасете абсолютные значения метрик будут другими.